

ACTIVITY TRACKING SYSTEM

Project Requirements & Design Document

Spring Boot REST API Training Project

1. Project Description

The Activity Tracking System is a web-based application that converts the current manual activity Excel sheet into a structured digital solution. Employees can record daily activities with date, start time, end time, activity type, activity subject and task description. The system automatically calculates hours, validates entries, generates monthly summaries and provides dashboard/reporting data.

2. Project Objectives

- Provide a simple and accurate way to log daily work activities.
- Reduce manual calculations and data-entry errors in the Excel process.
- Generate monthly summaries by Activity Type and Activity Subject.
- Provide dashboard data for hours, trends and totals.
- Use the project to practice Java, Spring Boot, REST APIs, JPA and PostgreSQL.

3. Internship Prerequisites

These are recommended prerequisites for a summer internship. Advanced experience is not required; students should have enough foundation to learn and complete the MVP within one month.

- Java basics: OOP, classes, interfaces, collections and exceptions.
- Basic Spring Boot: project structure, dependency injection and a simple REST controller.
- REST/HTTP basics: GET, POST, PUT, DELETE, JSON and status codes.
- Basic SQL/database concepts: tables, primary keys, foreign keys and simple queries.
- Git basics: clone, branch, commit, pull, push and basic merge/conflict awareness.
- Frontend basics: HTML, CSS and JavaScript. Previous React experience is useful but not mandatory.
- Required tools: JDK 17+, IDE, Maven, PostgreSQL, Git, Postman/Swagger, Node.js and npm.

Frontend recommendation: Use React + Vite for the internship. It is simpler and faster for a REST-driven internal dashboard. Next.js is acceptable only if a student already knows it; it is not required.

4. Scope

In Scope	Future / Optional
User management and roles	Advanced SSO / enterprise identity integration
Activity Types and Activity Subjects	Notifications
Daily Activity CRUD	Advanced approval workflow
Automatic duration calculation	Attachments
Overlap and time validation	Advanced analytics

Filtering and monthly summaries	Scheduled reports
Dashboard APIs	PDF export
Monthly Excel export	

5. User Roles

Role	Main Access
Employee	Create, view and update own activity entries; view own summary.
Manager	View team activities and summaries; approval can be enabled as an optional feature.
Admin	Manage users, Activity Types, Activity Subjects and system master data.

6. Core Functional Requirements

ID	Requirement
FR-01	Manage users and roles.
FR-02	Authenticate users using Spring Security/JWT.
FR-03	Manage Activity Types such as Project, Product, POC, Dev Plan, Meeting and Day Off.
FR-04	Manage Activity Subjects such as BM Microfocus, License Module, Microservices and other project/product/course names.
FR-05	Create, view, update and delete daily activity entries.
FR-06	Store activity date, start time, end time, type, subject and task description.
FR-07	Calculate activity duration automatically from start and end time.
FR-08	Prevent overlapping activities for the same user on the same date.
FR-09	Filter activities by user, date/month, type and subject.
FR-10	Calculate Total Days and Total Hours for the selected month.
FR-11	Generate hours grouped by Activity Type and Activity Subject.
FR-12	Provide dashboard APIs and monthly Excel export.

7. Main Business Rules

- End time must be later than start time.
- Duration is calculated by the backend.
- Overlapping activity entries for the same user and date are not allowed.
- Activity Type, Activity Subject and Task Description are required.
- Total Days = number of unique dates containing activity entries.
- Total Hours = sum of all activity durations for the selected period.
- Monthly summaries are grouped separately by Activity Type and Activity Subject.

8. High-Level Architecture

The solution follows a standard layered Spring Boot architecture: Web Client → REST Controller → Service → Repository → PostgreSQL. Cross-cutting concerns include security, validation, global exception handling and logging.

Layer	Responsibility
Web Client	Activity entry screens, dashboards and reports.
REST Controller	Expose HTTP endpoints and handle request/response DTOs.
Service	Business logic, validation, calculations and aggregation.
Repository	Database access using Spring Data JPA.
PostgreSQL	Persistent storage for users and activity data.

9. Main Data Model

Entity	Purpose
User	Employee/manager/admin information and role.
ActivityType	High-level category: Project, Product, POC, Dev Plan, Meeting, etc.
ActivitySubject	Specific work subject: project, product, course, POC or meeting name.
ActivityEntry	Daily logged activity including date, From/To, duration and description.
Approval / Comment	Optional extension for manager review and feedback.

10. Module Breakdown

Module	Scope	Backend Deliverables	Frontend Deliverables
User & Authentication	Users, roles and security	User APIs, login/JWT, role-based access.	Login, user management and protected routes.
Master Data	Activity Types & Subjects	CRUD APIs, entities, repositories and validation.	Master-data screens plus shared React foundation/API client.
Activity Entry	Core daily activity logging	CRUD, automatic duration, overlap validation and filters.	Activity form, list/table, edit/delete and filtering.
Dashboard & Reporting	Summaries and reporting	Monthly summary, grouped totals, dashboard APIs and Excel export.	Dashboard cards/charts, month filter and export UI.

11. Contribution / Work Distribution

Contributor	Assigned Module	Backend Tasks	Frontend & Integration
Ahmed Yehia	User & Authentication	User model, roles, login API, Spring Security, JWT and password encoding.	Login page, user management, protected routes/auth state; auth/error conventions and secured-flow testing.
Basem Emad	Master Data + Frontend Foundation	Activity Type/Subject models, repositories, CRUD APIs and validation.	React + Vite setup, routing/layout, shared API client, master-data screens and shared frontend conventions.
Daniel Akmal	Activity Entry	Activity CRUD, duration calculation, overlap validation, filtering and activity queries.	Activity form/list, edit/delete and filters; integrate User + Master Data APIs; business-logic tests.
Mostafa Haitham	Dashboard & Reporting	Monthly summary, grouped totals, dashboard APIs and Excel export.	Dashboard cards/charts, month filter, summary/export UI; final integration, reporting validation and demo.

12. Shared Responsibilities

- Agree API contracts and entity relationships before integration.
- Use feature branches and pull requests.
- Use common validation and global error-response conventions.
- Document endpoints using Swagger/OpenAPI.
- Write unit tests for important service logic.
- Review another contributor's work before merge.

13. Technology Stack

Java 17+ • Spring Boot • Spring Web • Spring Data JPA • PostgreSQL • Bean Validation • Spring Security/JWT • OpenAPI/Swagger • Apache POI for Excel export • Maven • Git

14. Suggested Delivery Plan

Phase	Deliverable
1	Project setup, database and common structure.
2	User & Authentication + Master Data.
3	Activity Entry and business validations.
4	Dashboard and monthly summaries.
5	Excel export, security completion and integration testing.
6	Final review, documentation and demo.

15. Definition of Done

- The feature works through REST APIs and is testable using Swagger/Postman.
- Validation and meaningful error handling are implemented.
- Important business logic has tests.
- Code follows Controller → Service → Repository separation.
- No hard-coded business data is used.
- Changes are reviewed and integrated without breaking other modules.
- README/API documentation is updated.

16. LLD References

The project is supported by four separate Low-Level Design diagrams:

- User & Authentication LLD
- Master Data LLD
- Activity Entry LLD
- Dashboard & Reporting LLD

17. One-Month Internship Delivery Plan

The internship duration is one month, so the project must remain an MVP. The four backend modules can be developed in parallel, while frontend work focuses only on the screens needed to demonstrate the complete flow.

Week	Target
Week 1	Project setup, PostgreSQL schema, API contracts, User/Auth and Master Data foundations.
Week 2	Activity Entry CRUD, duration calculation, validation and basic frontend forms.
Week 3	Dashboard summaries, filtering, frontend dashboard and backend/frontend integration.
Week 4	Excel export, testing, bug fixing, documentation, final integration and demo.

18. Frontend Recommendation

Recommended: React with Vite. For a one-month internship, it is simpler and faster than Next.js for this project because the application is mainly an internal dashboard consuming Spring Boot REST APIs. It avoids introducing server-side rendering, routing conventions and other Next.js concepts that are not required for the MVP.

Next.js is also acceptable if the assigned frontend contributor already knows it well. The backend remains technology-independent because integration is through REST/JSON.

Frontend	Recommendation
React + Vite	Preferred for the internship: simple setup, REST integration, forms, tables and dashboards.
Next.js	Use if the team already has experience with it; otherwise it adds unnecessary learning scope.
Other frontend	Acceptable if it can consume the documented REST APIs and deliver the required screens within one month.

19. MVP Screens

- Login
- Activity Entry form and activity list
- Activity Type / Activity Subject management
- Monthly Dashboard
- Monthly Report / Excel Export

20. High-Level Design (HLD)

The HLD presents the complete system architecture and shows how the web client, Spring Boot REST API, business modules and PostgreSQL database work together.

Activity Tracking / Log Activity Sheet System – High Level Design (HLD)

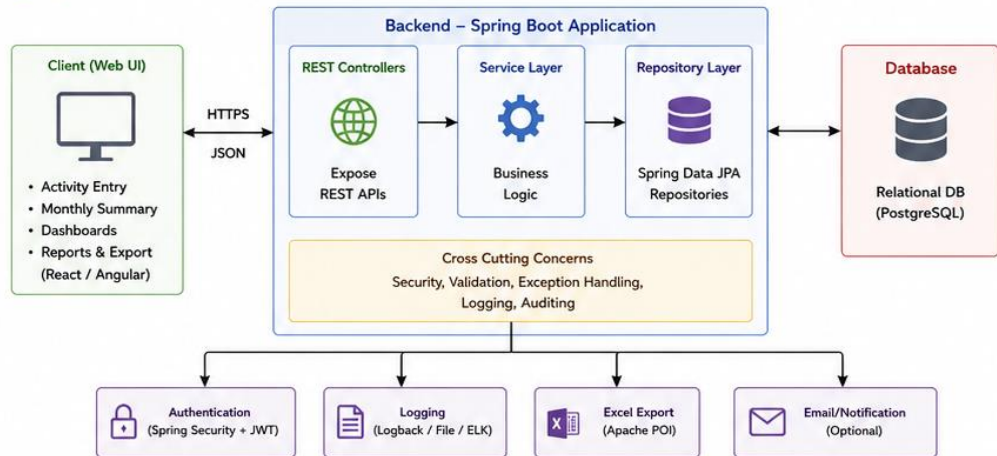
1 Overview

Activity Tracking System is a web application used to log daily work activities with start-end time and automatic hours calculation. It provides monthly summaries by Activity Type and Activity Subject (Project / Product / POC / Course / Meeting ...), dashboards, reports and export to Excel.

Key Features

- Log daily activities (From-To)
- Automatic hour calculation
- Activity Types (Project, Product, Dev Plan, Meetings, Day Off ...)
- Activity Subjects (Project / Module / POC / Course / Meeting ...)
- Monthly summary by Type & Subject
- Dashboards & reports
- Filtering, search & pagination
- Export to Excel

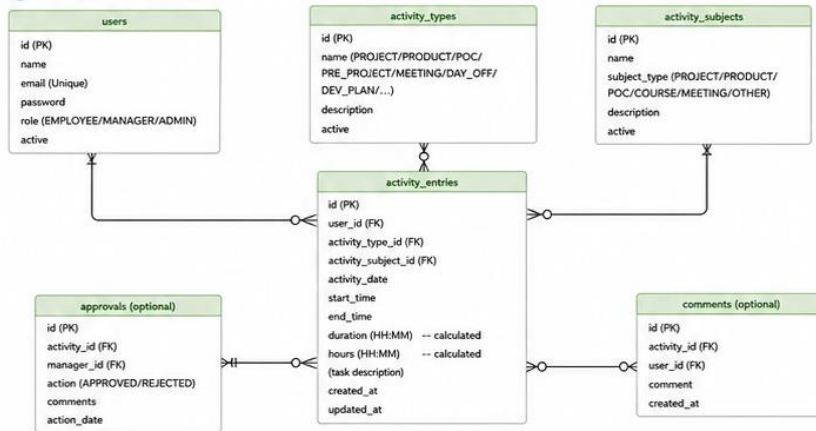
2 System Architecture



3 User Roles

- Employee / Intern**
 - Log daily activities
 - View own activities
 - View own summaries & dashboards
 - Export reports
- Manager**
 - View team activities
 - Approve / Reject activities (optional)
 - View team summaries & dashboards
 - Export reports
- Admin**
 - Manage users
 - Manage activity types
 - Manage activity subjects
 - System configuration

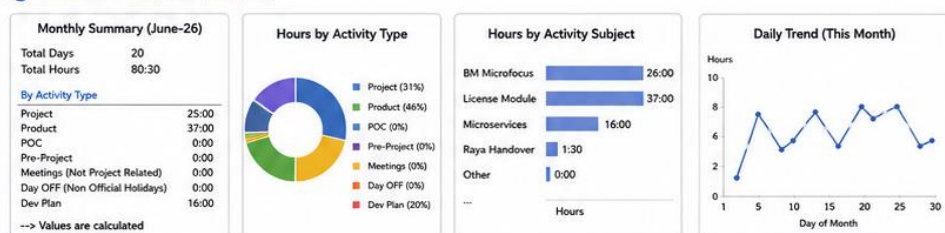
4 Data Model (ER Diagram)



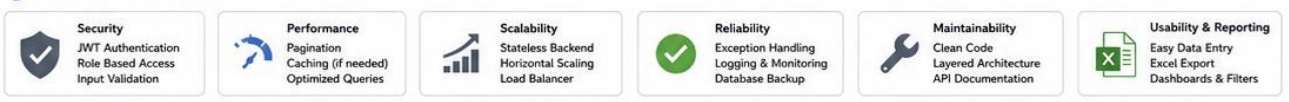
5 REST API (Examples)

Method	Endpoint	Description
POST	/api/auth/login	Login
GET	/api/users	Get Users
POST	/api/users	Create User
GET	/api/activity-types	Get Types
POST	/api/activity-types	Create Type
GET	/api/activity-subjects	Get Subjects
POST	/api/activity-subjects	Create Subject
POST	/api/activities	Create Activity
GET	/api/activities	Get Activities (filter)
GET	/api/activities/{id}	Get Activity
PUT	/api/activities/{id}	Update Activity
DELETE	/api/activities/{id}	Delete Activity
GET	/api/dashboard/summary	Monthly Summary
GET	/api/dashboard/by-type	Hours by Type
GET	/api/dashboard/by-subject	Hours by Subject
GET	/api/reports/export/monthly	Export Monthly Sheet (Excel)

6 Dashboards & Reports (Examples)



7 Non-Functional Requirements



8 Key Business Rules / Validations

- End time must be after start time
- Hours are calculated automatically
- Duration > 0
- No overlapping activities for the same user on the same day
- Activity Type is required
- Activity Subject is required
- Task description is required
- User can log within allowed working hours (configurable)
- Total Days = Count of unique dates
- Total Hours = Sum of all activity hours

Figure 1 - Activity Tracking System High-Level Design

21. Low-Level Design Diagrams

The following diagrams define the implementation-level design for the four main backend work packages.

Figure 2 - User & Authentication LLD

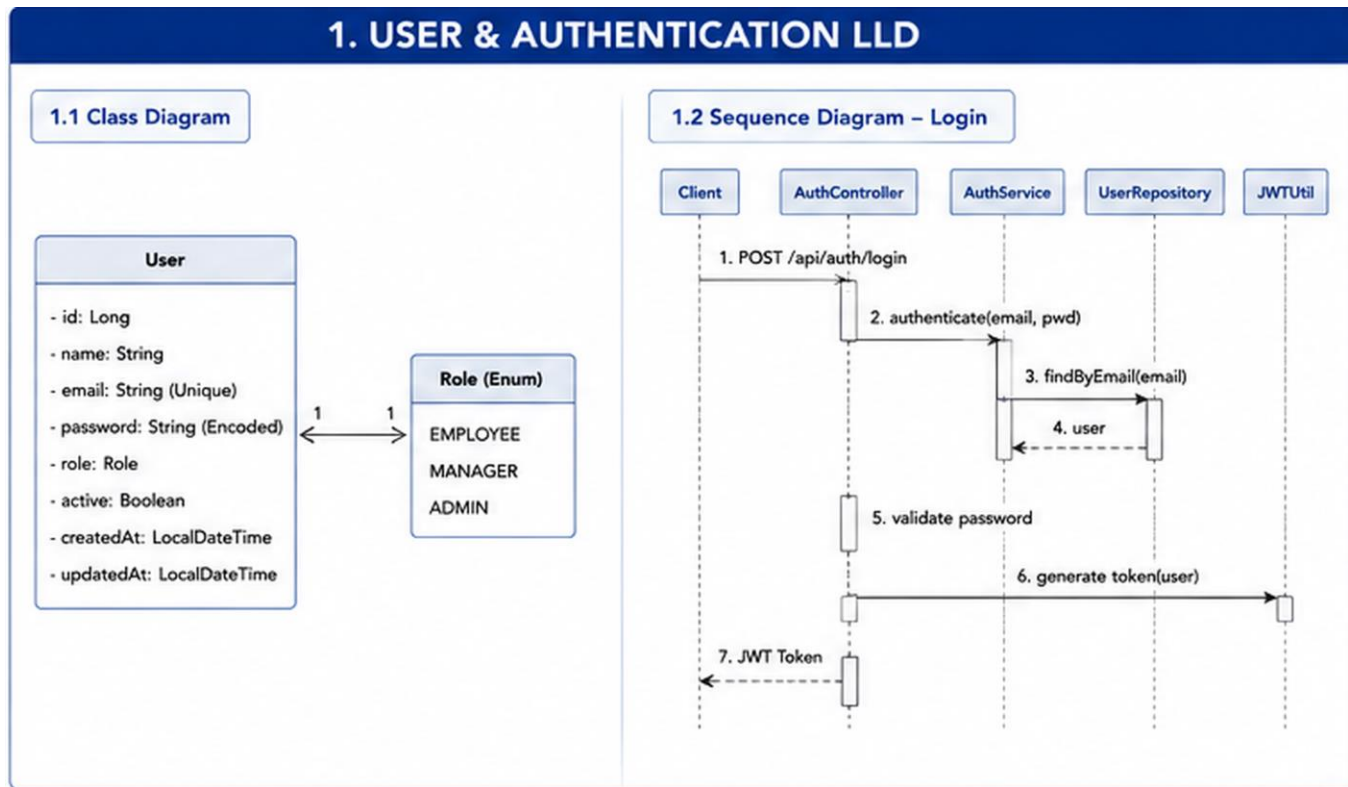


Figure 3 - Master Data LLD

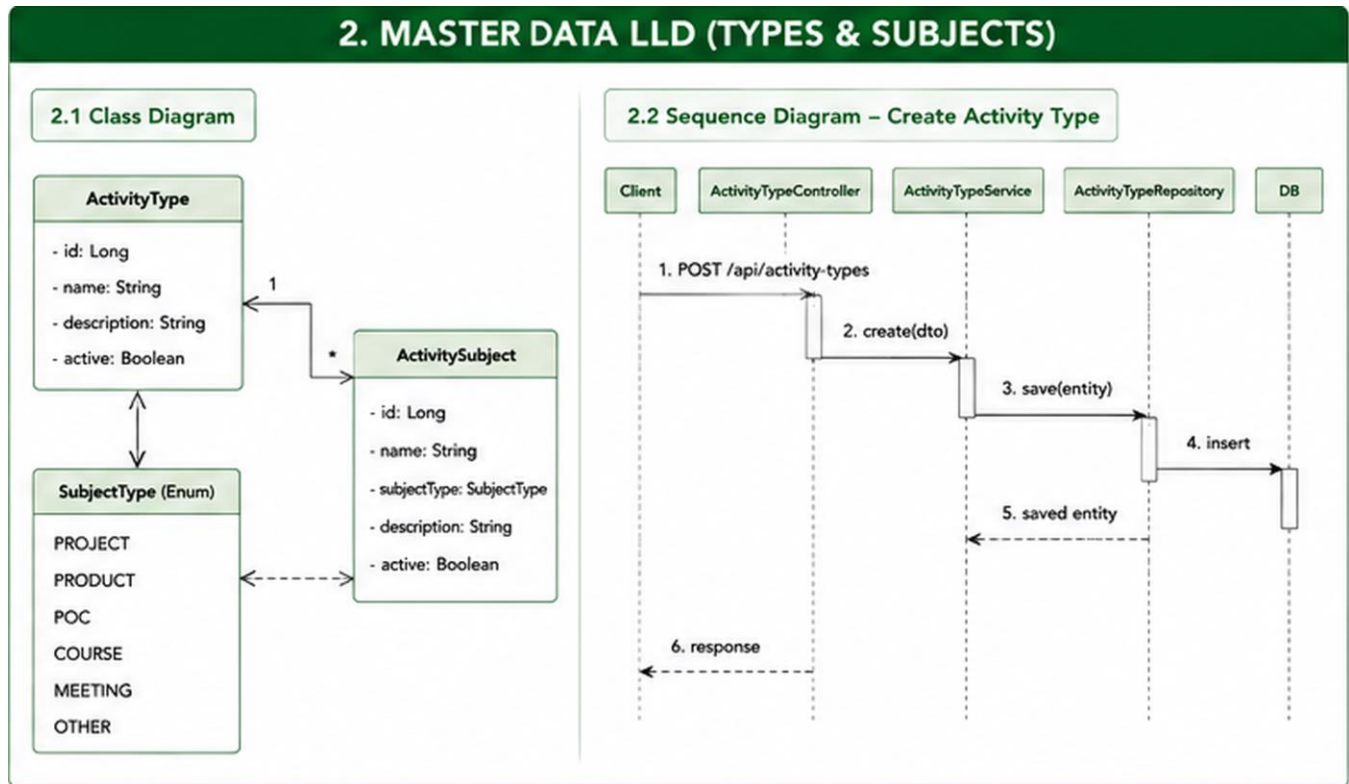


Figure 4 - Activity Entry LLD

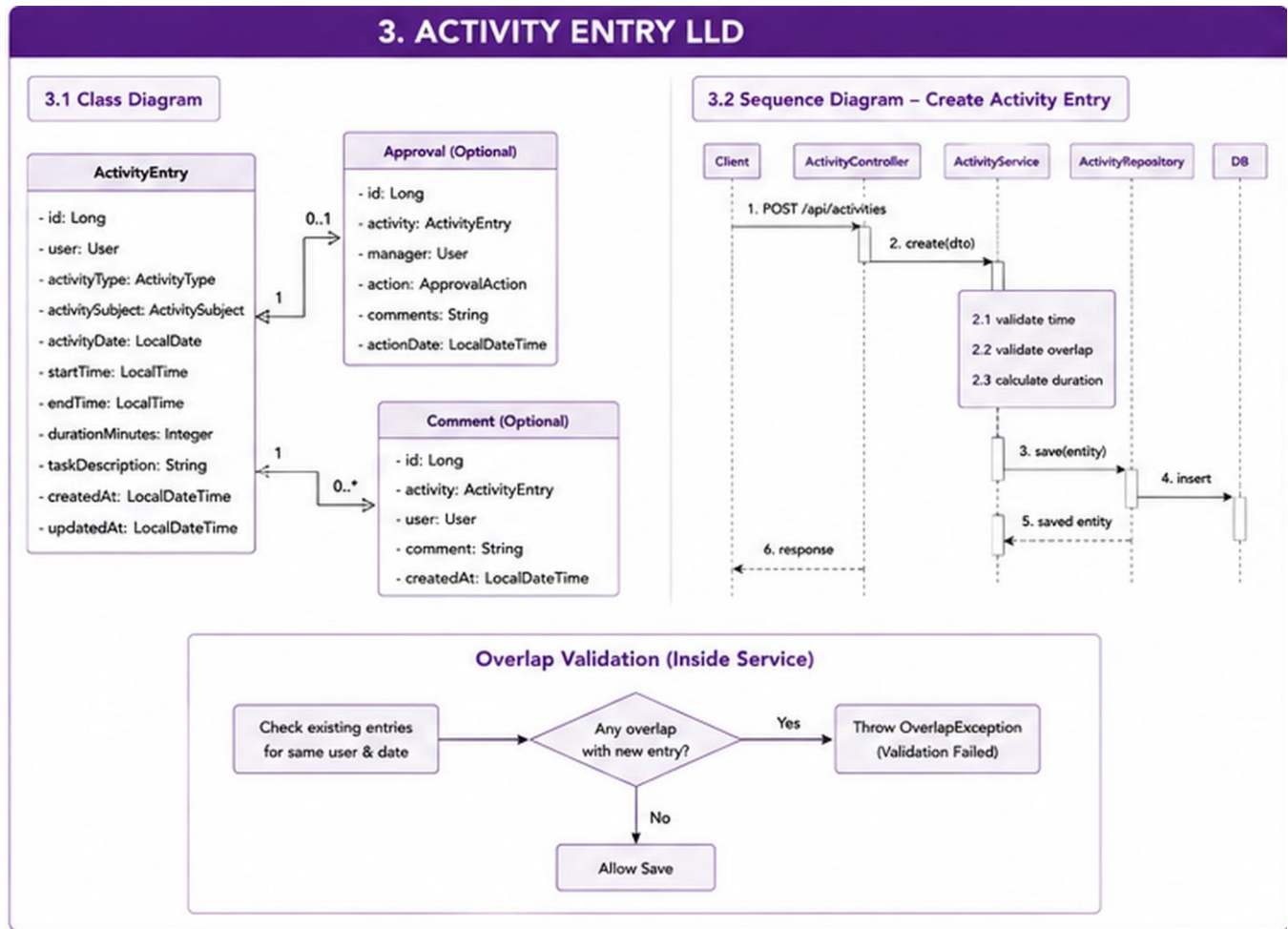
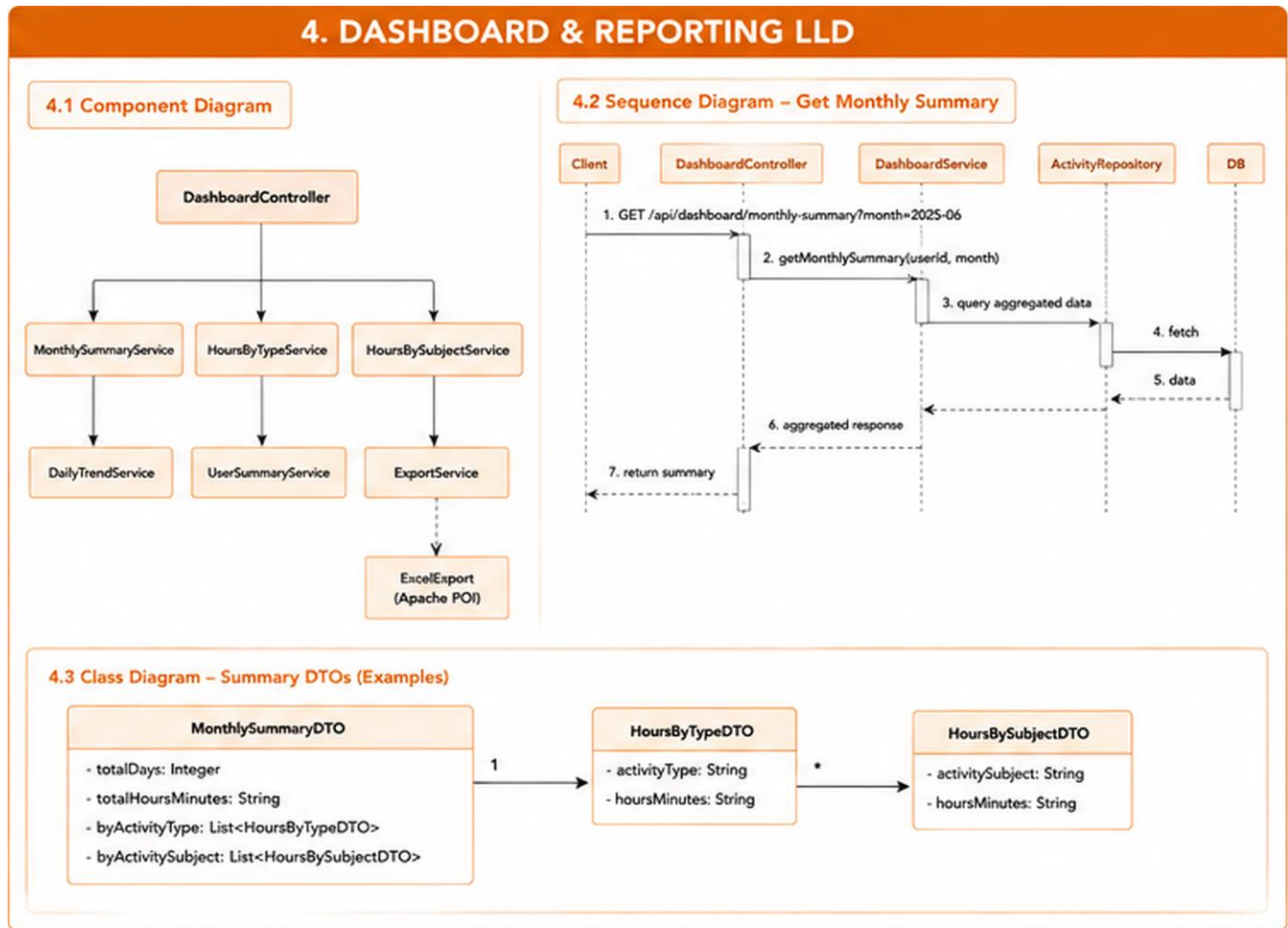


Figure 5 - Dashboard & Reporting LLD



Screen Requirements

The following screens define the minimum frontend scope for the MVP. Detailed UI mockups are not required; interns may choose the visual design as long as these functions are provided.

Screen	Minimum Requirements
Login	Email/password fields, validation, authentication, error handling and redirect after successful login.
Dashboard	Total Hours, Total Days, hours by Activity Type, hours by Activity Subject, simple charts and month filter.
My Activities	List logged activities; filter by date/type/subject; edit and delete actions; loading/empty/error states.
Add / Edit Activity	Date, From, To, Activity Type, Activity Subject and Task Description. Display calculated duration and validation errors.
Activity Types	List, create, edit and deactivate/delete Activity Types.
Activity Subjects	List, create, edit and deactivate/delete Activity Subjects.
Users	Admin view to list/create/edit users and assign roles.
Monthly Report	Display monthly summary and provide Excel export action.

Basic UI Guidelines

- Use one consistent application layout and navigation.
- Forms must clearly display validation errors returned by the frontend or backend.
- Tables/lists should handle loading, empty and error states.
- Dashboard charts should remain simple and readable.
- The application should work correctly on a normal laptop/desktop screen; advanced responsive design is not required.
- React + Vite is the recommended frontend stack for the one-month internship.

MVP Acceptance Criteria

Feature	Acceptance Criteria
Authentication	Valid user can log in; invalid credentials show an error; protected screens require authentication.
Master Data	Authorized user can create, view and update Activity Types and Activity Subjects; values are available in Activity Entry.
Activity Entry	User can create an activity; duration is calculated automatically; To <= From is rejected; overlapping entries are rejected.
Activity Management	Saved activities appear in the list and can be filtered, edited and deleted.
Dashboard	Selecting a month returns correct Total Days, Total Hours and grouped totals by type and subject.
Reporting	User can view a monthly summary and trigger an Excel export.
Integration	Frontend consumes Spring Boot REST APIs successfully and handles API validation/errors.
Quality	Important service business logic has tests; APIs are documented in Swagger/OpenAPI; project can be run using documented setup steps.

Final Internship Deliverables

- Working Spring Boot backend connected to PostgreSQL.
- Working React + Vite frontend integrated with the backend REST APIs.
- Database schema and sample/test data needed for demonstration.

- Swagger/OpenAPI documentation.
- Monthly dashboard and Excel export.
- Unit tests for important backend business logic.
- README with project setup and run instructions.
- Final integrated demo at the end of the one-month internship.